

Analysis of Algorithms

Assignment 1

Name: Ahmad Imran

SAP ID: 65171

Date: 05-10-2026

Q1. Finding Largest and Smallest Element

1. Strategy

I will go through the array only once. First I take the first element as both min and max. Then I check every other element one by one. If the element is greater than max, I put it in max. If not, then I check whether it is smaller than min, and if yes, I put it in min. At the end of the loop I have both the smallest and largest values.

2. Pseudocode

Algorithm MinMax(A, n)

Input: array A of n integers

Output: smallest and largest element of A

```
min ← A[0]
max ← A[0]
for i ← 1 to n - 1 do
    if A[i] > max then
        max ← A[i]
    else if A[i] < min then
        min ← A[i]
return min, max
```

Example: A = [7, 3, 9, 1, 5]

i	A[i]	A[i] > max	A[i] < min	min	max
–	–	–	–	7	7
1	3	No	Yes	3	7
2	9	Yes	–	3	9
3	1	No	Yes	1	9
4	5	No	No	1	9

So min = 1 and max = 9.

3. Primitive Operations

The main primitive operations in this algorithm are:

- (i) Assigning a value, e.g. $\text{min} \leftarrow A[0]$, $\text{max} \leftarrow A[i]$
- (ii) Indexing into the array, e.g. $A[i]$

- (iii) Comparing two numbers, e.g. $A[i] > \max$, $A[i] < \min$, $i \leq n - 1$
- (iv) Incrementing the counter i
- (v) Returning the result

4. Best Case and Worst Case

Counting operations:

$\min \leftarrow A[0]$ and $\max \leftarrow A[0] = 2 + 2 = 4$
 $i \leftarrow 1 = 1$
 loop test $i \leq n - 1$ runs n times $= n$
 $i \leftarrow i + 1$ runs $n - 1$ times (add + assign) $= 2(n - 1)$
 return $= 1$

Best case: when the array is in increasing order, e.g. [1, 2, 3, 4, 5]. Every time $A[i] > \max$ is true, so the else if part never runs. In each iteration we do: index, compare, index, assign = 4 operations.

$$T(n) = 4 + 1 + n + 2(n - 1) + 4(n - 1) + 1 = 7n$$

Worst case: when the array is in decreasing order, e.g. [5, 4, 3, 2, 1]. Every time the first check is false and the second check is true, so both comparisons happen and min is updated. In each iteration we do 6 operations.

$$T(n) = 4 + 1 + n + 2(n - 1) + 6(n - 1) + 1 = 9n - 2$$

If we only count comparisons between elements, best case $= n - 1$ and worst case $= 2(n - 1)$.

5. Asymptotic Notation

Best case: $7n \Rightarrow \Theta(n)$

Worst case: $9n - 2 \Rightarrow \Theta(n)$

So the running time is $O(n)$, $\Omega(n)$ and therefore $\Theta(n)$. We drop the constants. Also we have to look at every element at least once, so it can not be done in less than linear time.

6. Space Complexity

Only three extra variables are used (min, max and i). Their number does not change when n becomes bigger, so the extra space is $O(1)$. (The input array itself takes $O(n)$ space.)

Q2. Searching a Value X in an Array

1. Two Strategies

Method 1: Linear Search. Start from the first element and compare every element with X . If any element is equal to X , return its index. If the loop ends and X is not found, return -1 . It works on both sorted and unsorted arrays.

Method 2: Binary Search. This only works if the array is sorted. Compare X with the middle element. If equal, we found it. If X is bigger, then X can only be in the right half, so

we ignore the left half. If X is smaller, we ignore the right half. Every time the array becomes half.

2. Pseudocode

Algorithm LinearSearch(A, n, X)

Input: array A of n integers, value X

Output: index of X, or -1 if not found

```
for i ← 0 to n - 1 do
    if A[i] = X then
        return i
return -1
```

Algorithm BinarySearch(A, n, X)

Input: sorted array A of n integers, value X

Output: index of X, or -1 if not found

```
low ← 0
high ← n - 1
while low ≤ high do
    mid ← (low + high) / 2    {take integer part}
    if A[mid] = X then
        return mid
    else if A[mid] < X then
        low ← mid + 1
    else
        high ← mid - 1
return -1
```

Example: A = [2, 5, 8, 12, 16, 23, 38], X = 23

Step	low	high	mid	A[mid]	Result
1	0	6	3	12	12 < 23, low = 4
2	4	6	5	23	found at index 5

Binary search found it in 2 steps, linear search would take 6 steps.

3. Best Case and Worst Case

Linear Search

Best case: X is the first element, so only 1 comparison.

Worst case: X is the last element or not in the array, so n comparisons.

Binary Search

Best case: X is the middle element, so only 1 comparison.

Worst case: X is not in the array. The size keeps halving: n, n/2, n/4, ..., 1. This happens about $\log_2 n$ times, so comparisons $\approx \log_2 n + 1$.

e.g. $n = 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ is 4 steps and $\log_2 16 = 4$.

4. Big-O, Big-Ω and Big-Θ

	Linear Search	Binary Search
Best case	$\Theta(1)$	$\Theta(1)$
Worst case	$\Theta(n)$	$\Theta(\log n)$
Big-O	$O(n)$	$O(\log n)$
Big-Ω	$\Omega(1)$	$\Omega(1)$
Space	$O(1)$	$O(1)$

Big-O is the upper bound and Big-Ω is the lower bound. Θ can only be used when both are same. Here best and worst cases are different, so we write Θ separately for best case and worst case, not one Θ for the whole algorithm.

5. Comparison

For large n binary search is much faster. If $n = 1,000,000$ then linear search may need 1,000,000 comparisons, but binary search needs only about 20, because 2^{20} is about 1,000,000.

But binary search needs a sorted array. If the array is not sorted, we first have to sort it, which takes $O(n \log n)$. That is more than doing one linear search which is $O(n)$.

So:

- (i) Unsorted array and only one search: linear search is better.
- (ii) Sorted array: binary search is better.
- (iii) Unsorted array but many searches: sort it once and then use binary search every time.
- (iv) Very small array: linear search is simpler and the difference is very small.

6. Effect of Size and Arrangement of Data

Size: When n is small, both take almost same time. When n is large, the difference is very big because $\log n$ grows very slowly as compared to n . So for big data binary search is the better choice.

Arrangement: Linear search works on any order. Its time only depends on the position of X , if X is near the start it is fast and if it is at the end or missing it is slow. Binary search depends fully on the order because it only works on sorted data. If the data keeps changing a lot (insert/delete), then keeping it sorted is extra work, and linear search can be more practical.